

Relatório Técnico — Web Proxy com Controle de Conteúdo

Sistemas para Internet II
André Prisco Vargas
Antonella Cuello
169603

1. Introdução

Este trabalho teve como objetivo desenvolver um web proxy didático capaz de atuar como intermediário entre o navegador e a Internet. O sistema recebe uma URL, verifica se o domínio está bloqueado, realiza a requisição ao servidor de origem quando permitido, filtra palavras configuradas e registra os acessos em um arquivo de log.

2. Tecnologias escolhidas

As tecnologias utilizadas foram Python, Flask e Requests.

O Python foi escolhido por eu ter maior familiaridade, possuir sintaxe simples, boa legibilidade e bibliotecas adequadas para trabalhar com requisições HTTP e manipulação de arquivos. O Flask foi escolhido dada a sugestão do professor. Ele foi utilizado para criar o servidor web responsável por receber as requisições do usuário e controlar as rotas da aplicação. A biblioteca Requests foi usada para acessar os sites solicitados pelo usuário e obter suas respostas HTTP.

A escolha dessas tecnologias facilitou o desenvolvimento por permitir uma implementação direta e compreensível do fluxo do proxy.

3. Funcionamento do proxy

O proxy possui uma página inicial com um formulário onde o usuário informa a URL desejada. A partir disso, o sistema redireciona a requisição para a rota principal do proxy.

Ao receber a URL, o programa extrai o domínio usando `urlparse`. Em seguida, verifica se esse domínio está presente no arquivo `blocked.json`. Caso esteja, o sistema não realiza a requisição ao site e exibe uma página personalizada de bloqueio.

Se o site não estiver bloqueado, o proxy realiza a requisição usando `requests.get()`. Quando o conteúdo retornado é HTML, o sistema verifica as palavras configuradas no arquivo `words.json` e realiza substituições usando expressões regulares com busca case-insensitive. Quando o conteúdo não é HTML, como imagens ou arquivos binários, o proxy retorna os bytes do conteúdo junto com o status HTTP e o `Content-Type`.

4. Bloqueio de sites

O bloqueio de sites foi implementado a partir do arquivo `blocked.json`, que contém uma lista de domínios proibidos. Quando o domínio acessado está nessa lista, o proxy retorna uma página HTML personalizada informando que o acesso foi bloqueado.

Essa decisão permite que a lista de bloqueios seja alterada sem modificar diretamente o código Python.

5. Filtro de palavras

O filtro de palavras foi implementado com base no arquivo `words.json`. Cada entrada do arquivo possui uma palavra a ser procurada e sua respectiva substituição.

A filtragem ocorre apenas em conteúdos HTML. Essa decisão foi tomada para evitar problemas com arquivos binários, como imagens, PDFs ou outros tipos de conteúdo que não devem ser tratados como texto.

Também foi utilizada a opção `re.IGNORECASE`, permitindo que palavras com letras maiúsculas ou minúsculas sejam filtradas da mesma forma.

6. Logs de acesso

Cada requisição processada pelo proxy é registrada em um arquivo `log.txt`. O log contém a data e hora da requisição, a URL acessada e a ação executada pelo proxy: permitido, bloqueado ou filtrado.

Esse registro permite acompanhar o comportamento do sistema durante os testes.

7. Tratamento de erros

Foi implementado tratamento básico de erros utilizando `try/except`. Quando ocorre uma falha, como URL inválida, site indisponível ou conexão recusada, o sistema exibe uma página personalizada de erro em vez de mostrar mensagens técnicas diretamente ao usuário.

Essa decisão melhora a experiência de uso e deixa o sistema mais apresentável.

8. Como o proxy lida com HTTPS

O HTTPS apresenta uma limitação importante para proxies. Em um proxy real, o navegador geralmente utiliza o método CONNECT para criar um túnel criptografado entre o cliente e o servidor de destino. Nesse cenário, o proxy apenas encaminha dados criptografados e não consegue visualizar nem modificar o conteúdo HTML da página.

No projeto desenvolvido, algumas páginas HTTPS ainda podem ser modificadas porque a requisição é feita pela biblioteca Requests. Nesse caso, o Python estabelece a conexão HTTPS com o servidor, recebe o conteúdo jácriptografado e então o Flask devolve esse conteúdo ao navegador.

Entretanto, isso não representa o funcionamento completo de um proxy HTTPS real. Para filtrar conteúdo HTTPS em um proxy real, seria necessário interceptar a conexão TLS/SSL, o que exigiria certificados, configuração adicional no cliente e uma implementação mais complexa. Por esse motivo, o suporte completo a HTTPS não foi implementado.

9. Decisões de implementação

Algumas decisões importantes foram tomadas durante o desenvolvimento:

- criar páginas HTML personalizadas para bloqueio, erro e página inicial;
- retornar `response.content` em conteúdos não HTML para preservar arquivos binários;
- retornar o `Content-Type` original para que o navegador consiga interpretar corretamente imagens e outros arquivos;
- criar uma página inicial com formulário para melhorar a usabilidade do sistema;
- separar visualmente as páginas usando CSS.

10. Dificuldades encontradas

Uma das dificuldades foi compreender como o Flask recebe dados enviados pelo navegador. Foi necessário estudar o funcionamento de parâmetros de requisição, especialmente o uso de `request.args.get()`, utilizado para capturar a URL informada pelo usuário através do formulário da página inicial.

Outra dificuldade foi entender a importância dos headers HTTP. Sem o `Content-Type`, por exemplo, imagens eram exibidas como caracteres estranhos no navegador.

Durante a implementação do filtro de conteúdo, foi necessário estudar a função `re.sub()`. Inicialmente, a ideia era utilizar o método `replace()`, que já era conhecido por mim, porém ele diferencia letras maiúsculas e minúsculas. Como o trabalho exigia uma filtragem *case-insensitive*, foi necessário utilizar `re.sub()` com a opção `IGNORECASE`, permitindo substituir palavras independentemente da forma como foram escritas de maneira mais otimizada.

Também foi necessário compreender melhor a diferença entre um proxy didático feito com Flask e Requests e um proxy real de rede, especialmente no caso de conexões HTTPS.

11. Aprendizados

Durante o desenvolvimento, foram aprendidos conceitos importantes sobre o funcionamento da Web e do protocolo HTTP, como:

- estrutura de requisições e respostas HTTP;
- uso de status codes;
- importância dos headers;
- diferença entre conteúdo textual e binário;
- funcionamento básico de proxies;
- limitação de filtragem em conexões HTTPS;
- uso de rotas no Flask;

12. Uso de Inteligência Artificial

Durante o desenvolvimento, ferramentas de inteligência artificial foram utilizadas como apoio para esclarecimento de dúvidas, organização do projeto, explicação de conceitos e revisão de trechos de código.

A IA auxiliou principalmente na compreensão de Flask, Requests, manipulação de respostas HTTP, tratamento de erros e organização do README e do relatório. O código gerado ou sugerido foi analisado, testado e adaptado antes de ser incorporado ao projeto.

O uso da ferramenta contribuiu para o aprendizado, pois permitiu compreender melhor cada parte implementada e revisar decisões tomadas durante o desenvolvimento.

13. Conclusão

O projeto permitiu aplicar na prática conceitos estudados sobre HTTP, cliente-servidor e proxies. O sistema desenvolvido cumpre as principais funções propostas: repassar requisições, bloquear domínios, filtrar conteúdo HTML e registrar logs.

Além disso, o desenvolvimento ajudou a compreender limitações reais de proxies, especialmente em relação ao HTTPS, e reforçou a importância de headers, status codes e tratamento adequado de diferentes tipos de conteúdo.